

Initial Agent Documentation:

1. Project Overview

Project Name: VoiceAI Call Handling Agent

Purpose: A 24/7 AI-powered voice agent that answers inbound calls, qualifies leads, books appointments, answers FAQs, and transfers complex calls to humans.

Primary Target Customers:

- Clinics / Dental / Medical practices
- Real Estate agents
- Local service businesses (plumbing, HVAC, cleaning, etc.)
- Law firms & professional services

Key Benefits:

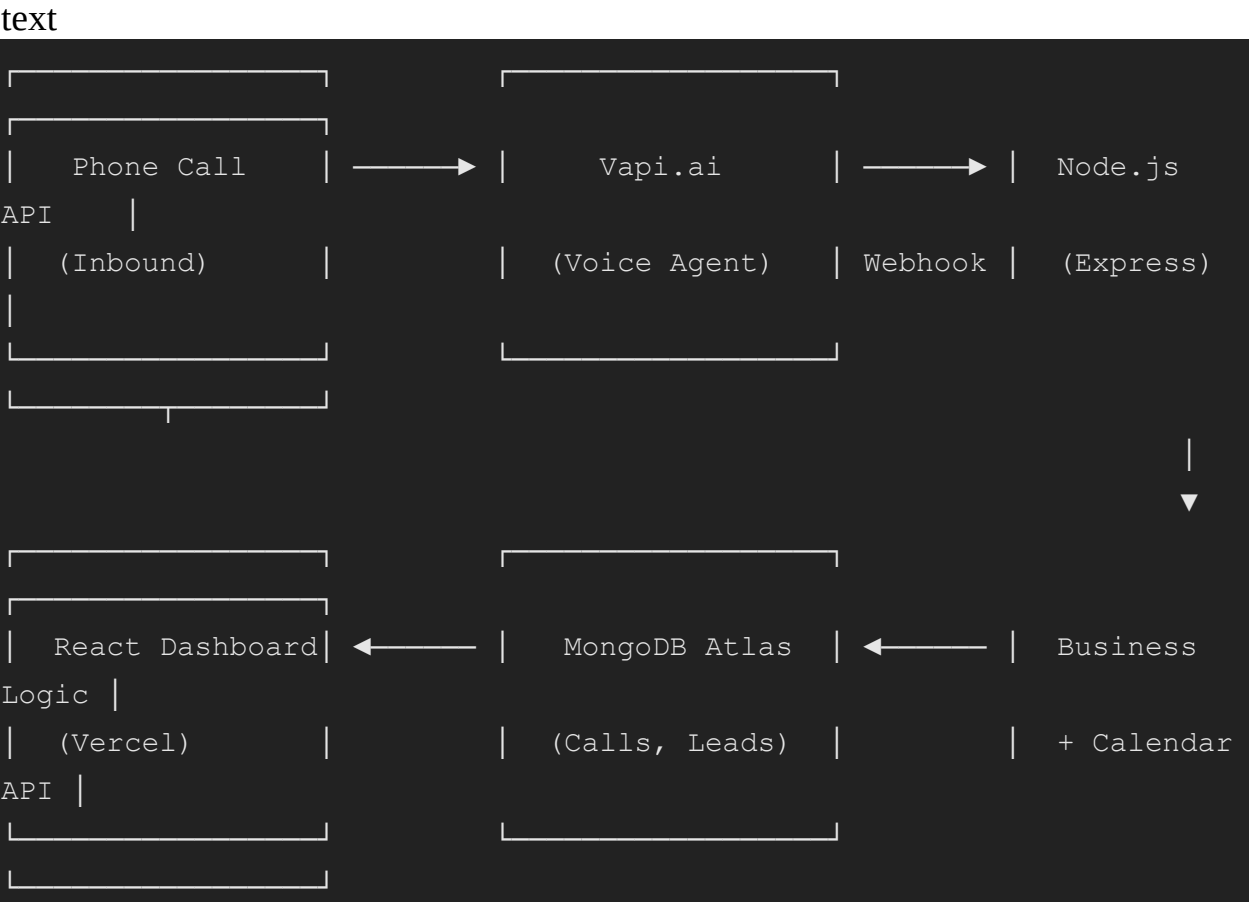
- Never miss a call
- Book appointments automatically
- Qualify leads before human intervention
- Reduce staffing costs
- Full call logs, transcripts & analytics

2. Tech Stack

Layer	Technology	Purpose
Frontend	React (Vite) + TypeScript	Dashboard for call logs, analytics, settings
Backend	Node.js + Express	API server, webhooks, business logic
Database	MongoDB Atlas	Store calls, transcripts, leads, appointments
Voice Platform	Vapi.ai	Speech-to-Text, LLM, Text-to-Speech, Telephony
Deployment	Vercel	Frontend + Backend (Serverless)
Authentication	JWT / NextAuth or Clerk (optional)	User login for dashboard

Calendar	Google Calendar / Calendly API	Appointment booking
CRM (Optional)	HubSpot / Salesforce / custom	Lead sync

3. System Architecture Diagram (Text + Mermaid)



Mermaid Version (paste into mermaid.live or compatible tools):

- Book Appointment
- Qualify Lead
- Transfer Call
- Inbound Phone Call
- Vapi Voice Agent

Function Call?

Node.js Backend

Human Agent

MongoDB

Google Calendar / CRM

Webhook Events

React Dashboard

4. Use Cases

Use Case 1: Appointment Booking (Clinic)

1. Patient calls clinic number
2. AI greets and asks reason for call
3. Collects name, phone, preferred date/time
4. Checks calendar availability via function call
5. Books appointment and confirms
6. Sends confirmation SMS/email

Use Case 2: Lead Qualification (Real Estate)

1. Prospect calls about a property
2. AI asks budget, timeline, preferred location
3. Scores lead (Hot / Warm / Cold)
4. Stores lead in MongoDB + CRM
5. Transfers hot leads to agent or schedules callback

Use Case 3: After-Hours Support

1. Customer calls outside business hours
2. AI answers FAQs from knowledge base
3. Takes message or books callback
4. Notifies owner via email/Slack

Use Case 4: Call Transfer

1. Complex question detected
 2. AI says “Let me connect you with a specialist”
 3. Warm transfer to human with context
-

5. Detailed Vapi Configuration Guide

Step 1: Create Vapi Account

- Go to <https://dashboard.vapi.ai>
- Sign up and get your **Private API Key** and **Public API Key**

Step 2: Create Assistant (Recommended Configuration)

Go to **Assistants** → **Create Assistant**

Basic Settings:

- Name: Clinic Receptionist (or your vertical)
- First Message:
Hello! Thank you for calling [Business Name]. This is Sarah, your AI assistant. How can I help you today?
- First Message Mode: assistant-speaks-first

Model (LLM):

JSON

```
{
  "provider": "openai",
  "model": "gpt-4o",
  "temperature": 0.4,
  "messages": [
    {
      "role": "system",
      "content": "You are a professional, friendly, and efficient receptionist for [Business Name]. Keep responses short (under 25 words when possible). Be polite and helpful. Always confirm important
```

```
details. If the user wants to book an appointment, collect: full name,
phone number, preferred date and time, and reason for visit. Use the
available tools when needed."
```

```
    }
  ]
}
```

Voice:

- Provider: 11labs or azure or vapi
- Recommended: 11labs → voiceId: 21m00Tcm4TlvDq8ikWAM (Rachel) or any natural female/male voice
- Stability: 0.5 | Similarity Boost: 0.75

Transcriber:

- Provider: deepgram
- Model: nova-2 or nova-3
- Language: en (or multi if needed)

Server URL (Webhook):

- Set to: <https://your-backend.vercel.app/api/vapi/webhook>
- This is critical for receiving call events and function calls.

Tools / Functions (Very Important):

Create these tools in Vapi Dashboard → Tools:

1. **bookAppointment**
2. **checkAvailability**
3. **saveLead**
4. **transferCall**

Example Function Schema for bookAppointment:

JSON

```
{
  "name": "bookAppointment",
  "description": "Book an appointment for the caller",
```

```

"parameters": {
  "type": "object",
  "properties": {
    "fullName": { "type": "string" },
    "phoneNumber": { "type": "string" },
    "preferredDate": { "type": "string", "description": "YYYY-MM-DD"
  },
    "preferredTime": { "type": "string" },
    "reason": { "type": "string" }
  },
  "required": ["fullName", "phoneNumber", "preferredDate",
"preferredTime"]
}
}

```

Step 3: Create / Import Phone Number

- Go to **Phone Numbers**
- Create Free Vapi Number or import from Twilio
- Assign the Assistant to the phone number (Inbound Settings)

Step 4: Server Messages (Webhook Events)

Enable these in Assistant → Advanced → Server Messages:

- status-update
- transcript
- function-call
- end-of-call-report
- hang
- speech-update

6. Environment Variables

Create .env file in backend and frontend:

Backend (.env)

env

```
PORT=5000
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/voiceai
VAPI_API_KEY=your_private_vapi_key
VAPI_PUBLIC_KEY=your_public_vapi_key
VAPI_ASSISTANT_ID=asst_xxxxxxxxxxxx
VAPI_PHONE_NUMBER_ID=pn_xxxxxxxxxxxx
JWT_SECRET=your_super_secret_jwt_key
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=
FRONTEND_URL=https://your-frontend.vercel.app
NODE_ENV=production
```

Frontend (.env)

env

```
VITE_API_URL=https://your-backend.vercel.app
VITE_VAPI_PUBLIC_KEY=your_public_vapi_key
```

7. MongoDB Database Schema

Collection: calls

JavaScript

```
{
  _id: ObjectId,
  vapiCallId: String,
  phoneNumber: String,
  direction: "inbound" | "outbound",
  status: "queued" | "ringing" | "in-progress" | "ended",
  startedAt: Date,
  endedAt: Date,
  duration: Number, // seconds
  transcript: String,
```

```
    recordingUrl: String,  
    summary: String,  
    leadScore: "hot" | "warm" | "cold",  
    metadata: Object,  
    createdAt: Date  
}
```

Collection: appointments

JavaScript

```
{  
  _id: ObjectId,  
  callId: ObjectId,  
  fullName: String,  
  phoneNumber: String,  
  email: String,  
  date: Date,  
  time: String,  
  reason: String,  
  status: "confirmed" | "cancelled" | "completed",  
  calendarEventId: String,  
  createdAt: Date  
}
```

Collection: leads

JavaScript

```
{  
  _id: ObjectId,  
  fullName: String,  
  phoneNumber: String,  
  email: String,  
  source: "voice-agent",  
  score: "hot" | "warm" | "cold",  
  notes: String,  
  callId: ObjectId,  
  createdAt: Date  
}
```

```
}
```

8. Backend API Endpoints (Node.js + Express)

Method	Endpoint	Description
POST	/api/vapi/webhook	Receive all Vapi events
GET	/api/calls	List all calls (with pagination)
GET	/api/calls/:id	Get single call details + transcript
GET	/api/appointments	List appointments
POST	/api/appointments	Create appointment manually
GET	/api/leads	List leads
GET	/api/analytics/overview	Dashboard stats
POST	/api/calls/outbound	Trigger outbound call
GET	/api/health	Health check

Example Webhook Handler (Node.js):

JavaScript

```
// routes/vapi.js
router.post('/webhook', async (req, res) => {
  const { message } = req.body;

  switch (message.type) {
    case 'status-update':
      // Update call status in MongoDB
      break;
    case 'function-call':
      // Handle bookAppointment, checkAvailability etc.
      // Return result to Vapi
      break;
    case 'end-of-call-report':
      // Save full transcript, summary, recording
      break;
  }
})
```

```
res.status(200).json({ received: true });  
});
```

9. Frontend Structure (React)

text

```
src/  
├── components/  
│   ├── CallList.tsx  
│   ├── CallDetail.tsx  
│   ├── AppointmentCalendar.tsx  
│   ├── LeadTable.tsx  
│   ├── StatsCards.tsx  
│   └── Sidebar.tsx  
├── pages/  
│   ├── Dashboard.tsx  
│   ├── Calls.tsx  
│   ├── Appointments.tsx  
│   ├── Leads.tsx  
│   └── Settings.tsx  
├── services/  
│   └── api.ts  
├── hooks/  
│   └── useCalls.ts  
└── App.tsx
```

10. Deployment on Vercel

1. Push frontend and backend to GitHub (can be monorepo or separate repos)
2. Import project in Vercel
3. Set all Environment Variables in Vercel Dashboard
4. For Backend: Set Root Directory to backend (if monorepo)
5. Add Custom Domain if needed
6. Update Vapi Server URL to your production backend URL

Important: Vercel serverless functions have timeout limits. For long-running webhooks, consider using Vercel Pro or move webhook handling to a more persistent service if needed.

11. Project Folder Structure (Recommended)

text

```
voice-ai-agent/
├── frontend/                # React App
│   ├── src/
│   ├── package.json
│   └── vercel.json
├── backend/                # Node.js Express
│   ├── src/
│   │   ├── routes/
│   │   ├── models/
│   │   ├── controllers/
│   │   ├── middleware/
│   │   └── index.js
│   ├── package.json
│   └── vercel.json
├── .env.example
└── README.md
```

12. MVP Development Roadmap

Phase	Tasks	Duration
1	Vapi Assistant + Basic Webhook + MongoDB	3–5 days
2	Appointment booking function + Calendar	3–4 days
3	React Dashboard (Calls + Appointments)	5–7 days
4	Lead scoring + Analytics	3 days
5	Authentication + Multi-tenant support	4–5 days
6	Polish + Testing + Launch	3–4 days

13. Pricing Suggestion for Selling

Plan	Price	Features
Starter	\$199/month	500 minutes + Basic Dashboard
Growth	\$399/month	1500 minutes + Calendar + CRM sync
Pro	\$799/month	Unlimited + Custom voice + Priority